

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

XÂY DỰNG GAME FLAPPY BIRD
TRÊN VI ĐIỀU KHIỂN STM32F429ZI

Học phần: Hệ nhúng – IT4210

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Nhóm sinh viên thực hiện: Nhóm saveEmbedded

Họ và tên	MSSV
Nguyễn Đình Thủy	20235437
Trương Minh Tâm	20235422
Tổng Trần Bình	20235275

Hà Nội, tháng 7 năm 2026

Lời mở đầu

Hệ thống nhúng hiện đại không chỉ điều khiển cảm biến và cơ cấu chấp hành mà còn có khả năng xử lý đồ họa, âm thanh và tương tác người dùng trong thời gian thực. Việc xây dựng một trò chơi trên vi điều khiển là bài toán tổng hợp phù hợp để vận dụng kiến thức về lập trình bare-metal, thư viện HAL, bộ định thời, ngắt, DMA, bộ nhớ ngoài và thiết kế phần mềm có ràng buộc tài nguyên.

Trong bài tập lớn này, nhóm xây dựng trò chơi Flappy Bird trên kit STM32F429I-Discovery. Hệ thống sử dụng LCD TFT 2.4 inch độ phân giải 240×320 , SDRAM ngoài để lưu hai framebuffer, bộ điều khiển cảm ứng STMPE811, nút nhấn vật lý, bộ khuếch đại I2S MAX98357 và loa ngoài. Toàn bộ phần mềm được phát triển bằng STM32CubeMX và STM32CubeIDE, sử dụng C cùng STM32 HAL/BSP.

Báo cáo trình bày kiến trúc phần cứng, tổ chức bộ nhớ hiển thị, cơ chế điều phối thời gian thực, máy trạng thái trò chơi, thuật toán vật lý và va chạm, cơ chế chuyển đổi framebuffer đồng bộ với VBLANK, cùng phương pháp tổng hợp và truyền âm thanh bằng DMA ở chế độ vòng.

Nhóm xin chân thành cảm ơn giảng viên hướng dẫn đã cung cấp kiến thức nền tảng và góp ý trong quá trình thực hiện. Do giới hạn thời gian và thiết bị đo, một số đánh giá thời gian thực mới dừng ở mức phân tích tĩnh và kiểm thử chức năng; các hạn chế này được nêu rõ trong phần cuối báo cáo.

Tóm tắt nội dung và đóng góp

Hệ thống hiện thực trò chơi Flappy Bird trên vi điều khiển STM32F429ZIT6 hoạt động ở 180 MHz. TIM6 tạo cơ sở thời gian với chu kỳ 10 ms; thuật toán vật lý và quá trình dựng khung hình được thực hiện sau mỗi hai chu kỳ định thời, tương ứng tốc độ danh định 50 FPS. Hình ảnh được tạo từ các đối tượng đồ họa cơ bản của BSP LCD trên framebuffer ARGB8888 trong SDRAM ngoài. Hai lớp LTDC được chuyển đổi tại khoảng nghỉ quét dọc nhằm hạn chế hiện tượng xé hình.

Người chơi có thể điều khiển bằng cảm ứng điện trở STMPE811 hoặc nút USER PA0. Cảm ứng được thăm dò mỗi 30 ms; nút vật lý dùng EXTI0 và chống dội bằng cửa sổ 80 ms. Game có menu, chọn nhân vật, chọn độ khó, bật/tắt nhạc và hiệu ứng, tạm dừng, chơi lại và bốn chủ đề màn chơi.

Âm thanh được tổng hợp trực tiếp ở tần số lấy mẫu 8 kHz. Nhạc nền sử dụng sóng vuông theo chuỗi nốt, trong khi hiệu ứng nhảy sử dụng sóng tam giác có tần số biến thiên. DMA1 Stream 5 truyền bộ đệm âm thanh nổi qua I2S3 ở chế độ vòng; chương trình phục vụ ngắt chỉ đánh dấu nửa bộ đệm cần cập nhật, còn quá trình sinh mẫu được thực hiện trong vòng lặp chính.

Các đóng góp kỹ thuật chính gồm:

1. Cấu hình clock 180 MHz, TIM6, UART, GPIO/EXTI và tích hợp BSP cho kit STM32F429I-Discovery.
2. Khởi tạo FMC/SDRAM, LTDC, ILI9341 và hai vùng framebuffer trong SDRAM ngoài.
3. Xây dựng máy trạng thái game, vật lý chim, sinh cột bằng LCG, phát hiện va chạm AABB và hệ thống điểm.
4. Xây dựng giao diện vector, nhiều skin/chủ đề và page flipping đồng bộ VBLANK.
5. Tích hợp cảm ứng STMPE811 qua I2C3 và nút nhấn EXTI có chống dội.
6. Tạo nhạc và hiệu ứng theo thời gian thực, truyền liên tục bằng I2S3/DMA circular.

Mục lục

Lời mở đầu	i
Tóm tắt nội dung và đóng góp	ii
Mục lục	iii
Danh mục hình vẽ	v
Danh mục bảng	vi
Danh mục thuật ngữ và từ viết tắt	vii
1 GIỚI THIỆU	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu đề tài	1
1.3 Nền tảng phần cứng và công cụ	2
1.4 Phạm vi và mã nguồn	2
1.5 Bố cục báo cáo	2
2 PHÂN CÔNG CÔNG VIỆC	3
2.1 Nguyên tắc phân chia	3
2.2 Các mốc tích hợp	3
2.3 Tập dùng chung và quy ước tích hợp	4
3 THIẾT KẾ PHẦN CỨNG	5
3.1 Sơ đồ khối tổng thể	5
3.2 Cấu hình clock hệ thống	5
3.3 Phân bổ chân và ngoại vi	6
3.4 TIM6 và nút nhấn ngoài	6
3.5 SDRAM, LTDC và LCD	7
3.5.1 Tổ chức bộ nhớ framebuffer	7
3.5.2 Quét LCD và page flipping	7
3.6 Cảm ứng STMPE811	8
3.7 I2S3, DMA và MAX98357	8
4 THIẾT KẾ VÀ HIỆN THỰC PHẦN MỀM	9

4.1	Kiến trúc module	9
4.2	Trình tự khởi động và superloop	9
4.3	Máy trạng thái game	11
4.4	Vật lý chim và điều khiển nhảy	12
4.5	Sinh cột, độ khó và chủ đề	12
4.6	Phát hiện va chạm và tính điểm	13
4.7	Renderer vector và double buffering	13
4.8	Xử lý cảm ứng	13
4.9	Bộ tổng hợp âm thanh	14
5	KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ	15
5.1	Kết quả chức năng	15
5.2	Kết quả biên dịch và tài nguyên	15
5.3	Phân tích thời gian đáp ứng	16
5.4	Các khó khăn kỹ thuật	16
5.4.1	Dependency của BSP LCD/SDRAM	16
5.4.2	Framebuffer vượt SRAM nội	16
5.4.3	Xé hình và đồng bộ VBLANK	17
5.4.4	Tọa độ cảm ứng phụ thuộc orientation	17
5.4.5	Cân bằng ISR và superloop	17
5.5	Hạn chế của phiên bản hiện tại	17
6	KẾT LUẬN	18
6.1	Kết quả đạt được	18
6.2	Hướng phát triển	18
6.3	Lời cảm ơn	18
A	KỊCH BẢN CHỤP ẢNH VÀ NGHIỆM THU	19
A.1	Danh sách cảnh cần ghi nhận	19
A.2	Khung ảnh sẽ dùng trong báo cáo	20
A.3	Quy tắc trình bày ảnh	22
	Tài liệu tham khảo	23

Danh sách hình vẽ

3.1	Sơ đồ khối phần cứng của hệ thống	5
3.2	Cây clock rút gọn của hệ thống	5
3.3	Ảnh xạ hai framebuffer trong SDRAM ngoài	7
3.4	Cơ chế nạp luân phiên hai nửa buffer âm thanh	8
4.1	Kiến trúc phân lớp của firmware	9
4.2	Trình tự khởi động firmware	10
4.3	Máy trạng thái của game	11
4.4	Pipeline dựng và hiển thị một khung hình	13
4.5	Luồng dữ liệu âm thanh	14
A.1	Mô hình phần cứng hoàn chỉnh của hệ thống	20
A.2	Cây clock cấu hình trên STM32CubeMX	21
A.3	Cấu hình chân vi điều khiển trên STM32CubeMX	21
A.4	Kết quả biên dịch thành công trên STM32CubeIDE	21
A.5	Màn hình menu chính trên LCD	22
A.6	Giao diện trong khi chơi FlappyBird	22

Danh sách bảng

1.1	Nền tảng phần cứng và công cụ phát triển	2
2.1	Phân công thành viên	3
2.2	Trách nhiệm trong các tệp dùng chung	4
3.1	Các chân giao tiếp ứng dụng chính	6
3.2	Luồng dữ liệu giữa các khối phần cứng	6
3.3	Bố trí hai framebuffer trong SDRAM	7
4.1	Phân chia tác vụ và ngữ cảnh thực thi	11
4.2	Các chuyển trạng thái quan trọng của game	12
5.1	Ma trận chức năng đã hiện thực	15
5.2	Kích thước firmware sau biên dịch	15
5.3	Chu kỳ danh định của các hoạt động thời gian thực	16
A.1	Kịch bản chụp ảnh và ảnh màn hình nghiệm thu	19
A.1	Kịch bản chụp ảnh và ảnh màn hình nghiệm thu (tiếp theo)	20

Danh mục thuật ngữ và từ viết tắt

Thuật ngữ	Ý nghĩa
ADC	Analog-to-Digital Converter – bộ chuyển đổi tương tự sang số.
AABB	Axis-Aligned Bounding Box – hộp bao song song với các trục, dùng để kiểm tra va chạm.
BSP	Board Support Package – lớp phần mềm hỗ trợ phần cứng theo từng kit.
DMA	Direct Memory Access – truyền dữ liệu trực tiếp giữa ngoại vi và bộ nhớ.
EXTI	External Interrupt/Event Controller – bộ điều khiển ngắt ngoài.
FMC	Flexible Memory Controller – bộ điều khiển bộ nhớ ngoài của STM32F429.
Framebuffer	Vùng nhớ chứa giá trị màu của toàn bộ khung hình.
HAL	Hardware Abstraction Layer – lớp trừu tượng phần cứng của STM32Cube.
I2C	Inter-Integrated Circuit – bus nối tiếp hai dây.
I2S	Inter-IC Sound – giao tiếp nối tiếp cho âm thanh số.
LCG	Linear Congruential Generator – bộ sinh số giả ngẫu nhiên tuyến tính.
LTDC	LCD-TFT Display Controller – bộ điều khiển LCD TFT tích hợp.
NVIC	Nested Vectored Interrupt Controller – bộ điều khiển ngắt của Cortex-M.
PLL	Phase-Locked Loop – mạch nhân/chia xung nhịp.
SDRAM	Synchronous Dynamic Random-Access Memory – RAM động đồng bộ ngoài.
VBLANK	Vertical blanking – khoảng nghỉ giữa hai lần quét khung hình.

1. GIỚI THIỆU

1.1 Đặt vấn đề

Flappy Bird có luật chơi đơn giản nhưng đặt ra nhiều yêu cầu điển hình của một hệ thống nhúng đa phương tiện: cập nhật vật lý theo chu kỳ cố định, phản hồi đầu vào với độ trễ thấp, dựng lại toàn bộ màn hình, hạn chế xé hình, phát âm thanh liên tục và sử dụng bộ nhớ hiệu quả. Trên STM32F429, các yêu cầu này phải được giải quyết trong giới hạn 2 MB Flash, 256 kB SRAM nội và không có hệ điều hành.

Màn hình 240×320 ở định dạng ARGB8888 cần $240 \times 320 \times 4 = 307\,200$ byte cho một khung hình. Chỉ một framebuffer đã vượt quá dung lượng của một bank SRAM chính, còn hai framebuffer cần 614 400 byte. Vì vậy, việc khai thác SDRAM 8 MB gắn trên board là điều kiện cần để thực hiện double buffering.

Âm thanh cũng yêu cầu luồng dữ liệu đều đặn. Nếu CPU tự ghi từng mẫu vào I2S, thời gian dành cho game và đồ họa sẽ bị phân mảnh. DMA circular giải quyết vấn đề truyền liên tục, nhưng phần mềm vẫn phải nạp kịp hai nửa buffer và tránh làm quá nhiều việc trong ISR.

1.2 Mục tiêu đề tài

Đề tài hướng tới các mục tiêu sau:

- Xây dựng game Flappy Bird chạy độc lập trên STM32F429I-Discovery, không cần máy tính khi vận hành.
- Duy trì nhịp cập nhật ổn định khoảng 50 FPS bằng timer phần cứng.
- Hiển thị đồ họa mượt trên LCD ILI9341, sử dụng SDRAM và double buffering.
- Nhận thao tác từ màn hình cảm ứng và nút nhấn vật lý có chống dôi.
- Phát nhạc nền và hiệu ứng đồng thời qua MAX98357 bằng I2S và DMA.
- Tổ chức mã nguồn thành các module phần cứng, game, âm thanh và input có giao diện rõ ràng.

1.3 Nền tảng phần cứng và công cụ

Bảng 1.1: Nền tảng phần cứng và công cụ phát triển

Thành phần	Mô tả
Vi điều khiển	STM32F429ZIT6, ARM Cortex-M4F, xung tối đa 180 MHz, Flash 2 MB.
Kit phát triển	STM32F429I-Discovery với LCD TFT 2.4 inch, SDRAM 8 MB và cảm ứng STMPE811.
Âm thanh	MAX98357, nhận dữ liệu I2S và khuếch đại class-D cho loa ngoài.
IDE	STM32CubeIDE, GNU Tools for STM32, ngôn ngữ C11.
Cấu hình ngoại vi	Các tham số được thiết lập trong STM32CubeMX và lưu trong tệp <code>FlappyBird.ioc</code> ; lệnh Generate Code sinh mã khởi tạo và cấu trúc dự án cho STM32CubeIDE.
Thư viện	STM32 HAL, CMSIS và BSP STM32F429I-Discovery.

1.4 Phạm vi và mã nguồn

Mã nguồn được tổ chức thành các mô-đun ứng dụng chính gồm `main.c`, `game.c`, `audio.c`, trình điều khiển LCD/SDRAM và trình điều khiển cảm ứng STMPE811. Quá trình phát triển và quản lý phiên bản được thực hiện tại [kho mã nguồn GitHub của nhóm](#).

Project không sử dụng TouchGFX hoặc RTOS. Giao diện được vẽ trực tiếp bằng BSP LCD, còn lịch trình được điều phối bằng superloop, TIM6, SysTick và các cờ ngắt. Báo cáo không đánh giá chất lượng âm thanh bằng thiết bị chuyên dụng và không đưa ra số liệu FPS đo bằng oscilloscope khi chưa có bằng chứng thực nghiệm.

1.5 Bố cục báo cáo

Sau phần giới thiệu, Chương 2 trình bày phân công công việc. Chương 3 phân tích thiết kế phần cứng và các ngoại vi. Chương 4 mô tả kiến trúc phần mềm, thuật toán trò chơi và cơ chế âm thanh. Chương 5 tổng hợp kết quả biên dịch, mức sử dụng tài nguyên, các vấn đề kỹ thuật và đánh giá hệ thống. Chương 6 trình bày kết luận và hướng phát triển.

2. PHÂN CÔNG CÔNG VIỆC

2.1 Nguyên tắc phân chia

Nhóm phân chia hệ thống thành ba nhóm chức năng có giao diện tương đối độc lập: nền tảng phần cứng và hiển thị, logic trò chơi và đồ họa, âm thanh và thiết bị đầu vào. Các tệp dùng chung như `main.c` và `stm32f4xx_it.c` được tích hợp theo phạm vi chức năng nhằm hạn chế xung đột khi quản lý phiên bản.

Bảng 2.1: Phân công thành viên

Thành viên	MSSV	Vai trò	Nhiệm vụ chính
Nguyễn Đình Thủy	20235437	Phần cứng và hiển thị	Clock 180 MHz, GPIO, UART, TIM6/EXTI; FMC/SDRAM; LTDC, ILI9341, framebuffer và các thành phần HAL/BSP liên quan.
Trương Minh Tâm	20235422	Logic và đồ họa	Máy trạng thái; vật lý chim; sinh cột; phát hiện va chạm; giao diện vector và chuyển đổi framebuffer.
Tổng Trần Bình	20235275	Âm thanh và đầu vào	I2S3, DMA1 Stream 5, tổng hợp âm thanh; I2C3/STMPE811; thăm dò cảm ứng và xử lý nút nhấn.

2.2 Các mốc tích hợp

Quá trình phát triển được tổ chức theo các mốc sau:

1. Thiết lập clock và ngoại vi trong STM32CubeMX, lưu cấu hình tại `FlappyBird.ioc`, sau đó sử dụng Generate Code để sinh dự án STM32CubeIDE.
2. Tích hợp BSP Discovery, FMC/SDRAM và LTDC; kiểm tra LCD cùng hai vùng framebuffer.
3. Xây dựng module game, máy trạng thái, vật lý, va chạm và đồ họa vector.
4. Tích hợp I2S/DMA, bộ tổng hợp nhạc/hiệu ứng và chân shutdown của MAX98357.
5. Tích hợp cảm ứng, nút EXTI, chống dội và các màn hình cài đặt.
6. Biên dịch toàn hệ thống, bổ sung đầy đủ các thành phần phụ thuộc của HAL/BSP và kiểm tra mức sử dụng tài nguyên từ tệp ELF.

2.3 Tập dùng chung và quy ước tích hợp

Bảng 2.2: Trách nhiệm trong các tập dùng chung

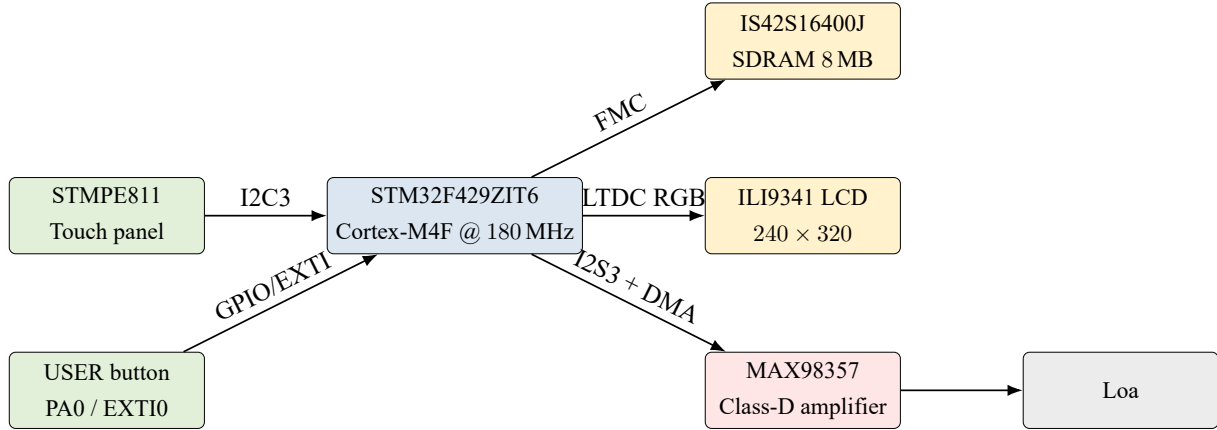
Tệp	Nội dung tích hợp
Core/Src/main.c	Khởi tạo phần cứng, LCD, audio, game, touch; superloop; callback TIM6 và EXTI.
Core/Inc/main.h	Định nghĩa chân BIRD_BUTTON PA0 và AMP_SD PG2.
stm32f4xx_hal_msp.c	Clock ngoại vi, GPIO alternate function và NVIC cho TIM6/UART.
stm32f4xx_it.c	ISR EXTI0, TIM6_DAC và DMA1_Stream5.
stm32f4xx_hal_conf.h	Bật các module HAL cần cho DMA2D, SDRAM, I2C, LTDC, SPI, TIM và UART.

Mọi thay đổi được biên dịch kiểm tra trước khi đẩy lên kho mã nguồn. Đối với các tệp do STM32CubeMX sinh tự động, mã bổ sung được đặt trong các vùng USER CODE để được bảo toàn khi thực hiện Generate Code lại.

3. THIẾT KẾ PHẦN CỨNG

3.1 Sơ đồ khối tổng thể

Hình 3.1 trình bày các khối chức năng chính và giao tiếp tương ứng của hệ thống.



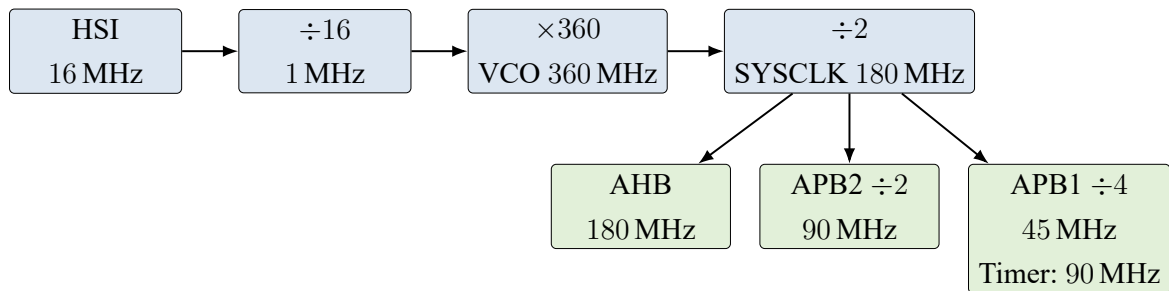
Hình 3.1: Sơ đồ khối phần cứng của hệ thống

STM32F429 đóng vai trò điều phối trung tâm. LTDC quét framebuffer trong SDRAM ra LCD mà không cần CPU ghi từng pixel theo thời gian quét. DMA1 truyền mẫu âm thanh từ SRAM tới SPI3 ở chế độ I2S. CPU tập trung cập nhật logic, vẽ buffer sau và sinh mẫu cho nửa buffer âm thanh vừa phát xong.

3.2 Cấu hình clock hệ thống

Project dùng HSI 16 MHz làm nguồn PLL. Các tham số PLLM=16, PLLN=360 và PLLP=2 tạo SYSCLK:

$$f_{SYSCLK} = \frac{16 \text{ MHz}}{16} \times \frac{360}{2} = 180 \text{ MHz.} \quad (3.1)$$



Hình 3.2: Cây clock rút gọn của hệ thống

AHB chạy ở 180 MHz; APB1 chia 4 còn 45 MHz, nhưng clock timer APB1 là 90 MHz; APB2 chia 2 còn 90 MHz. Firmware bật Over-Drive và cấu hình Flash latency 5 để bảo đảm

hoạt động ở tần số tối đa.

3.3 Phân bổ chân và ngoại vi

Bảng 3.1: Các chân giao tiếp ứng dụng chính

Chức năng	Chân MCU	Cấu hình
Nút nhảy	PA0	GPIO input, ngắt sườn lên EXTI0.
AMP_SD	PG2	GPIO output, mức cao sau khi audio khởi tạo.
UART1 TX/RX	PA9 / PA10	AF7, 115200-8-N-1, dùng debug.
I2S3 WS	PA15	AF6 SPI3, word select trái/phải.
I2S3 CK	PB3	AF6 SPI3, bit clock.
I2S3 SD	PC12	AF6 SPI3, dữ liệu âm thanh tới MAX98357.
Touch SCL	PA8	AF4 I2C3.
Touch SDA	PC9	AF4 I2C3.
LCD RGB/control	Nhiều chân A–G	AF14/AF9 LTDC theo BSP Discovery.
SDRAM	Nhiều chân B–G	FMC Bank 2 theo sơ đồ board.

Bảng 3.2: Luồng dữ liệu giữa các khối phần cứng

Khối nguồn	Khối đích	Vai trò trong hệ thống
STMPE811	CPU qua I2C3	Cung cấp tọa độ chạm cho menu và lệnh nhảy.
CPU/DMA2D	SDRAM	Dựng khung hình mới trong back buffer.
SDRAM	LTDC/LCD	LTDC đọc liên tục front buffer để quét RGB ra panel.
CPU	DMA1/I2S3	Nạp mẫu nhạc và hiệu ứng vào buffer vòng.
I2S3	MAX98357/loa	Chuyển mẫu số thành âm thanh khuếch đại.

3.4 TIM6 và nút nhấn ngoài

TIM6 nhận clock 90 MHz. Với prescaler 8999 và period 99, tần số update là:

$$f_{update} = \frac{90\,000\,000}{(8999 + 1)(99 + 1)} = 100 \text{ Hz}, \quad (3.2)$$

tương ứng một tick mỗi 10 ms. ISR TIM6_DAC_IRQHandler gọi HAL, sau đó callback tăng `game_tick_count`. Vòng lặp game chỉ cập nhật sau hai tick nên có chu kỳ danh định 20 ms.

Nút USER PA0 tạo ngắt EXTI0. Callback không gọi trực tiếp logic game mà chỉ đặt cờ `jump_requested`. Cách này giữ ISR ngắn và tránh vẽ LCD hoặc phát âm thanh trong ngữ cảnh ngắt. Hai lần nhận hợp lệ phải cách nhau tối thiểu 80 ms, nhờ đó lọc phần lớn rung tiếp điểm cơ học.

3.5 SDRAM, LTDC và LCD

3.5.1 Tổ chức bộ nhớ framebuffer

SDRAM IS42S16400J được ánh xạ từ địa chỉ `0xD0000000`, kích thước 8 MB. Hai layer LTDC dùng định dạng ARGB8888:

Bảng 3.3: Bố trí hai framebuffer trong SDRAM

Buffer	Địa chỉ đầu	Kích thước ảnh	Ghi chú
Front	<code>0xD0000000</code>	307,200 byte	Đang được LTDC quét.
Back	<code>0xD0050000</code>	307,200 byte	CPU/DMA2D vẽ khung kế tiếp.



Hình 3.3: Ánh xạ hai framebuffer trong SDRAM ngoài

Offset `0x50000`=327,680 byte lớn hơn kích thước một ảnh, do đó hai vùng không chồng lấn. Tổng dữ liệu pixel thực tế là 614,400 byte, chỉ chiếm khoảng 7.32% SDRAM ngoài.

3.5.2 Quét LCD và page flipping

BSP cấu hình LTDC cho panel ILI9341 với vùng hoạt động 240×320 . Mỗi frame, game chọn layer sau, vẽ background, cột, chim, HUD và panel giao diện. Khi hoàn tất, firmware dùng các hàm `NoReload` để đổi trạng thái hiển thị của hai layer và yêu cầu `HAL_LTDC_Reload(..., LTDC_RELOAD_VERTICAL_BLANKING)`. Địa chỉ hiển thị chỉ đổi tại VBLANK nên giảm đường cắt ngang do LTDC đọc đúng vùng mà CPU đang sửa.

DMA2D được BSP LCD sử dụng cho các thao tác fill/copy. So với vòng lặp CPU ghi từng pixel, ngoại vi này giảm thời gian tô các hình chữ nhật lớn như nền trời, mặt đất và thân cột.

3.6 Cảm ứng STMPE811

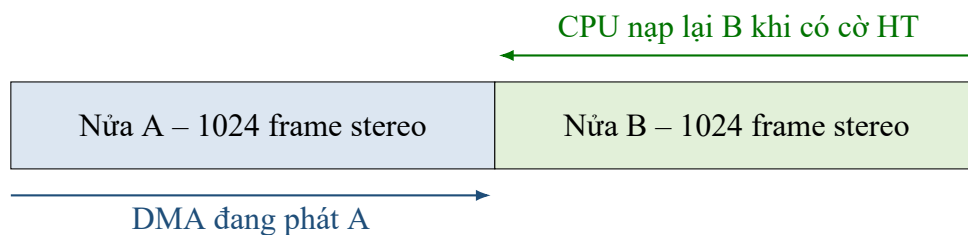
STMPE811 giao tiếp qua I2C3, địa chỉ 8-bit $0x82$. Hàm `BSP_TS_Init` cài driver, thiết lập biên theo kích thước LCD và khởi động bộ đo touch. Superloop đọc trạng thái mỗi 30 ms, đủ nhanh cho menu nhưng không chiếm bus I2C liên tục.

Driver BSP thực hiện hiệu chỉnh, đổi trục và lọc sai khác nhỏ của tọa độ. Vì hướng lắp panel và LCD có thể khác nhau giữa revision board, `Game_TouchHitsButton` kiểm tra thêm các phép đảo và hoán đổi trục. Đây là giải pháp chịu sai khác orientation tốt, dù về lâu dài nên thay bằng một ma trận hiệu chỉnh duy nhất lưu theo board.

3.7 I2S3, DMA và MAX98357

SPI3 được chuyển sang I2S master transmit 16-bit. PLLI2S dùng $N = 192$, $R = 2$; thanh ghi prescaler I2S được đặt 187 để tạo tốc độ gần 8 kHz. Dữ liệu mono sau khi tổng hợp được chép vào cả kênh trái và phải.

Buffer DMA có 4096 phần tử `uint16_t`, tương đương 8192 byte hay 2048 frame stereo. DMA1 Stream 5 chạy memory increment, peripheral/memory half-word, circular và phát ngắt half-transfer/transfer-complete. Khi audio đã sẵn sàng, PG2 được kéo cao để bỏ shutdown MAX98357; việc giữ PG2 thấp trong lúc khởi tạo giúp hạn chế tiếng bụp ở loa.



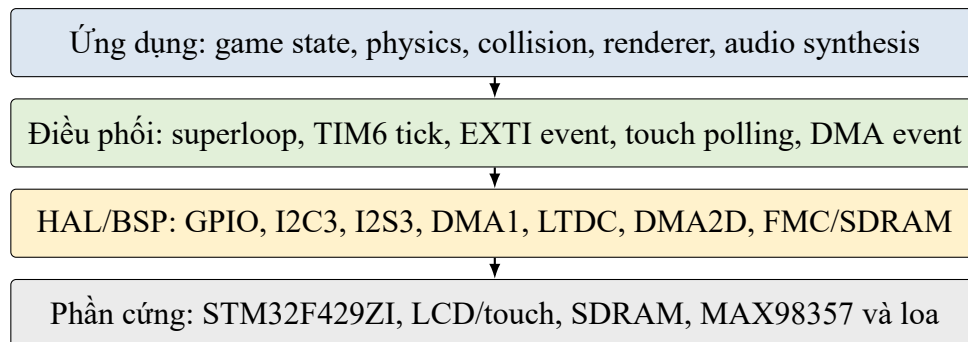
Hình 3.4: Cơ chế nạp luân phiên hai nửa buffer âm thanh

4. THIẾT KẾ VÀ HIỆN THỰC PHẦN MỀM

4.1 Kiến trúc module

Phần mềm được chia thành bốn lớp chính:

- **Khởi tạo và điều phối:** `main.c`, TIM6, SysTick, callback EXTI và superloop.
- **Game/renderer:** `game.c` quản lý trạng thái, vật lý, cột, va chạm, giao diện và page flip.
- **Audio:** `audio.c` cấu hình I2S/DMA, tổng hợp nhạc/hiệu ứng và nạp buffer.
- **BSP/driver:** LCD, SDRAM, ILI9341, touch và STMPE811.

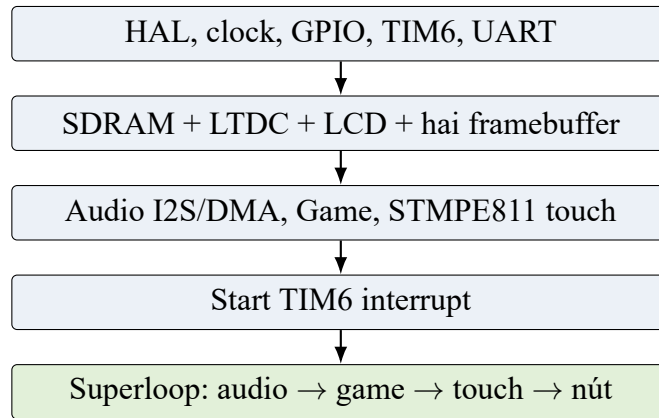


Hình 4.1: Kiến trúc phân lớp của firmware

Các API ứng dụng được giữ nhỏ: `Game_Init`, `Game_Update`, `Game_Jump`, `Game_Touch` và nhóm hàm `Audio_*`. Nhờ đó `main.c` không phụ thuộc vào cấu trúc nội bộ của chim, cột hoặc bộ dao động âm thanh.

4.2 Trình tự khởi động và superloop

Sau `HAL_Init` và `SystemClock_Config`, firmware khởi tạo GPIO, TIM6, UART, sau đó thực hiện lần lượt LCD/SDRAM, hai layer, audio, game và touch. TIM6 chỉ được start sau khi các module đã sẵn sàng.

**Hình 4.2:** Trình tự khởi động firmware

Đoạn rút gọn dưới đây cho thấy nguyên tắc điều phối không chặn:

Listing 4.1: Điều phối tác vụ trong superloop

```
Audio_Update();

if ((game_tick_count - last_game_tick) >= GAME_FRAME_TICKS) {
    last_game_tick = game_tick_count;
    Game_Update();
    Audio_Update();
}

if (touch_available &&
    ((HAL_GetTick() - last_touch_poll) >= TOUCH_POLL_MS)) {
    BSP_TS_GetState(&touch_state);
    /* Edge/repeat filtering, then Game_Touch(x, y). */
}

if (jump_requested) {
    jump_requested = 0U;
    Game_Jump();
}
```

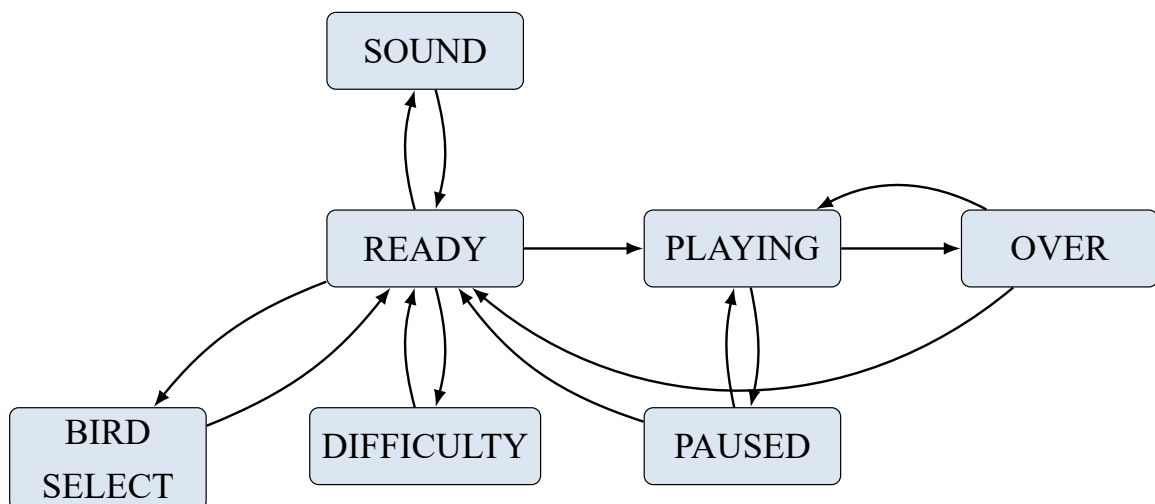
Bảng 4.1: Phân chia tác vụ và ngữ cảnh thực thi

Tác vụ	Ngữ cảnh	Chu kỳ/sự kiện	Công việc chính
TIM6 callback	ISR	10 ms	Tăng bộ đếm tick, không xử lý game.
Game update	Main	20 ms	Vật lý, va chạm, dựng frame và page flip.
Touch scan	Main	30 ms	Đọc I2C, lọc cạnh/giữ và phát sự kiện menu.
Audio DMA	ISR	HT/TC	Xóa cờ và đánh dấu nửa buffer cần nạp.
Audio synth	Main	Theo cờ DMA	Sinh mẫu, trộn, clip và ghi buffer.

Superloop không dùng `HAL_Delay`; do đó audio có thể được nạp bất cứ khi nào DMA báo cờ, còn touch và game chạy đúng chu kỳ riêng.

4.3 Máy trạng thái game

Game định nghĩa bảy trạng thái. `READY` là menu chính; ba trạng thái `BIRD_SELECT`, `DIFFICULTY` và `SOUND` là các màn cài đặt; `PLAYING` chạy vật lý; `PAUSED` dừng cập nhật; `OVER` hiển thị kết quả.

**Hình 4.3:** Máy trạng thái của game

Bảng 4.2: Các chuyển trạng thái quan trọng của game

Trạng thái	Sự kiện	Trạng thái kế	Hành động
READY	Start/Jump	PLAYING	Reset điểm, chim, cột và bắt đầu nhạc.
PLAYING	Pause	PAUSED	Giữ nguyên khung chơi và dừng cập nhật vật lý.
PAUSED	Continue	PLAYING	Tiếp tục vòng chơi hiện tại.
PLAYING	Collision	OVER	Phát hiệu ứng, cập nhật high score.
OVER	Retry	PLAYING	Khởi tạo một vòng chơi mới.
OVER/PAUSED	Menu	READY	Trở lại menu chính.
READY	Bird/Difficulty/Sound	Màn cài đặt	Hiển thị và cập nhật tùy chọn tương ứng.

Mỗi lần chuyển trạng thái, firmware dừng trước cả buffer đang hiển thị và buffer sau khi cần, tránh việc frame đầu tiên của màn mới chứa dữ liệu cũ.

4.4 Vật lý chim và điều khiển nhảy

Tọa độ theo pixel được lưu bằng số thực để chuyển động mượt. Với mỗi frame:

$$v_{k+1} = v_k + 0.34, \quad (4.1)$$

$$y_{k+1} = y_k + v_{k+1}. \quad (4.2)$$

Khi nhảy, vận tốc được đặt về -5.4 pixel/frame. Trục y hướng xuống nên vận tốc âm làm chim đi lên. Với nhịp 50 FPS, hiệu ứng nhảy kéo dài nhiều frame mà không cần delay. Nhấn nút ở READY hoặc OVER cũng bắt đầu vòng chơi mới.

4.5 Sinh cột, độ khó và chủ đề

Hệ thống duy trì hai cột. Khoảng trống có chiều cao 102 pixel; vị trí dọc được sinh trong miền an toàn bằng LCG:

$$x_{n+1} = 1664525x_n + 1013904223 \pmod{2^{32}}. \quad (4.3)$$

Khi một cột đi khỏi cạnh trái, nó được đưa ra sau cột xa nhất và nhận gap mới. Ba mức

độ khó thay đổi khoảng cách cột: EASY 138, MEDIUM 118 và HARD 102 pixel. Tốc độ cột bắt đầu ở 2.10 pixel/frame, tăng 0.10 sau mỗi episode và giới hạn ở 2.50 pixel/frame.

Mỗi 5 điểm mở một episode mới. Bốn theme SUNNY DAY, NIGHT SKY, GREEN FOREST và SNOW LAND thay đổi màu trời, đất, cột và HUD. Người dùng cũng có thể chọn BIRD, DUCK, PLANE hoặc EAGLE; các skin sử dụng cùng hitbox để giữ luật chơi nhất quán.

4.6 Phát hiện va chạm và tính điểm

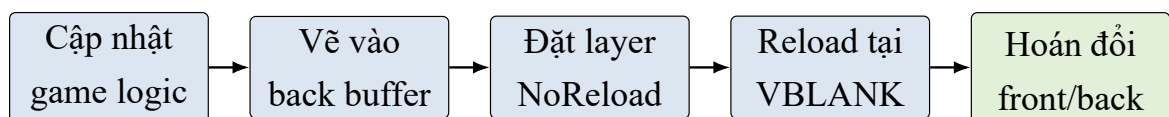
Chim có hitbox hình chữ nhật tại $x = 55$, kích thước xấp xỉ 22×14 pixel. Với mỗi cột, chương trình trước hết kiểm tra hai khoảng theo trục x . Nếu giao nhau, chim chỉ an toàn khi toàn bộ hitbox nằm trong khoảng gap. Đây là kiểm tra AABB, không cần căn bậc hai hoặc lượng giác.

Va chạm cũng xảy ra nếu chim chạm vùng HUD phía trên hoặc mặt đất ở $y = 290$. Điểm chỉ tăng một lần khi mép phải cột đi qua vị trí chim; cờ `scored` ngăn cộng lặp. `high_score` được giữ trong RAM cho tới khi reset MCU.

4.7 Renderer vector và double buffering

Game không dùng bitmap lớn. Cảnh nền, mây, núi, cột, mặt đất, chim và nút được tạo từ hình chữ nhật, đường thẳng, vòng tròn, ellipse và font BSP. Phương án này tiết kiệm Flash, cho phép đổi theme bằng bảng màu và phù hợp với DMA2D fill.

Mỗi frame được vẽ vào back layer. Sau khi vẽ xong, hai layer được đổi trạng thái visible bằng `NoReload` rồi reload tại `VBLANK`. Firmware chờ cờ `VBR` tối đa 25 ms; nếu quá thời gian thì reload ngay để tránh deadlock. Sau đó chỉ số front/back được hoán đổi.



Hình 4.4: Pipeline dựng và hiển thị một khung hình

4.8 Xử lý cảm ứng

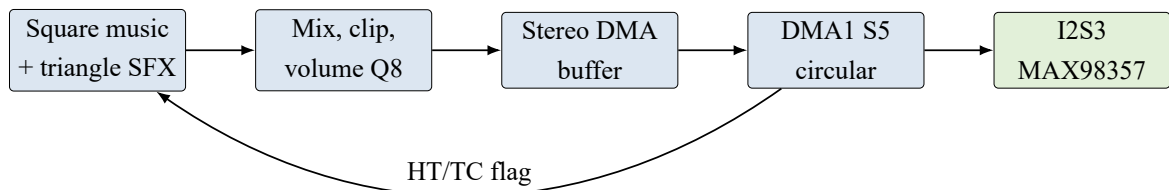
Cảm ứng được đọc mỗi 30 ms. Một lần chạm mới được nhận ngay; khi giữ tay, thao tác có thể lặp sau 160 ms, hữu ích cho nút mũi tên chọn skin/độ khó. Khi thả tay, cờ `touch_was_down` được xóa.

Các nút menu dùng hitbox hình chữ nhật. Tùy trạng thái, `Game_Touch` ánh xạ vùng chạm sang Start, Bird, Difficulty, Sound, Pause, Retry, Menu, Continue hoặc Cancel. Trong `PLAYING`, chạm ngoài nút Pause được hiểu là lệnh nhảy.

4.9 Bộ tổng hợp âm thanh

Nhạc nền là chuỗi 12 phần tử `AudioNote`, mỗi nốt chứa số mẫu và phase step. Bộ tích lũy pha 32-bit tạo sóng vuông. Hiệu ứng nhảy dùng sóng tam giác; tần số và biên độ giảm theo `jump_remaining`, tạo tiếng lướt ngắn. Hai nguồn được cộng, giới hạn trong $[-12000, 12000]$ rồi nhân hệ số âm lượng Q8.

ISR DMA chỉ đọc cờ HT/TC, xóa cờ phần cứng và đặt bit pending tương ứng. `Audio_Update` sao chép rồi xóa bitmap pending trong critical section rất ngắn, sau đó sinh mẫu ở main context. Thiết kế này tránh thực hiện vòng lặp tổng hợp hàng nghìn mẫu trong ISR và giảm ảnh hưởng tới TIM6/touch.



Hình 4.5: Luồng dữ liệu âm thanh

5. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

5.1 Kết quả chức năng

Các chức năng đã hiện thực của hệ thống được tổng hợp trong Bảng 5.1.

Bảng 5.1: Ma trận chức năng đã hiện thực

Nhóm chức năng	Kết quả
Khởi động phần cứng	Clock 180 MHz, GPIO, UART, TIM6, LCD/SDRAM, audio và touch được khởi tạo theo trình tự.
Giao diện	Menu, chọn skin, độ khó, âm thanh, pause, game over và episode banner.
Gameplay	Trọng lực, nhảy, cột tái sinh, sinh gap, tính điểm, high score, va chạm AABB.
Hiển thị	ARGB8888, hai layer SDRAM, page flip VBLANK, primitive vector và DMA2D.
Input	Touch polling 30 ms, repeat 160 ms, nút PA0 EXTI debounce 80 ms.
Âm thanh	Nhạc nền/sfx tổng hợp, volume, bật/tắt độc lập, I2S3 và DMA circular.

5.2 Kết quả biên dịch và tài nguyên

Kích thước các vùng bộ nhớ của tệp ELF sau biên dịch được xác định bằng công cụ `arm-none-eabi-size`:

Bảng 5.2: Kích thước firmware sau biên dịch

text	data	bss	Tổng thập phân	Tổng hex
43,948	240	11,048	55,236	0xD7C4

Ước lượng Flash là $text + data = 44\,188$ byte, khoảng 2.11% của 2 MB. Dữ liệu tĩnh trong SRAM là $data + bss = 11\,288$ byte, khoảng 4.31% của 256 kB. Hai framebuffer không nằm trong BSS vì dùng địa chỉ tuyệt đối trong SDRAM; chúng chiếm thêm 614,400 byte bộ nhớ ngoài. Buffer âm thanh 8192 byte là thành phần lớn nhất trong SRAM ứng dụng.

Phạm vi của số liệu

Số liệu trên được lấy từ tệp ELF trong thư mục Debug. Kết quả phản ánh kích thước bộ nhớ tĩnh và không thay thế phép đo mức sử dụng stack cục đại, tải CPU, tốc độ khung hình hoặc độ dao động chu kỳ trên phần cứng.

5.3 Phân tích thời gian đáp ứng**Bảng 5.3:** Chu kỳ danh định của các hoạt động thời gian thực

Hoạt động	Chu kỳ	Nhận xét
TIM6 tick	10 ms	Cơ sở thời gian phần cứng.
Game update/render	20 ms	Mục tiêu 50 FPS.
Touch polling	30 ms	Độ trễ tối đa khoảng một chu kỳ polling.
Touch hold repeat	160 ms	Tránh lặp quá nhanh trong menu.
Button debounce	80 ms	Bỏ qua cạnh EXTI quá gần nhau.
DMA half-buffer	Khoảng 128 ms	1024 frame stereo ở 8 kHz.

Khoảng thời gian nạp nửa buffer audio khá rộng so với vòng lặp game. Tuy nhiên, thời gian vẽ toàn màn hình phụ thuộc tốc độ SDRAM và DMA2D; cần đo DWT hoặc GPIO để xác nhận worst-case vẫn nhỏ hơn 20 ms.

5.4 Các khó khăn kỹ thuật**5.4.1 Dependency của BSP LCD/SDRAM**

Driver BSP không phải một file độc lập. LCD kéo theo `lcd.h`, `fonts.h`, các font source, DMA2D và LTDC; SDRAM kéo theo HAL SDRAM cùng LL FMC; driver Discovery còn sử dụng SPI/I2C. Nếu chỉ sao chép ba file BSP, compiler báo thiếu header, unknown type và linker thiếu symbol. Cách khắc phục là đưa đủ header/source và bật module tương ứng trong `stm32f4xx_hal_conf.h`.

5.4.2 Framebuffer vượt SRAM nội

Hai framebuffer ARGB8888 cần 600 KiB, lớn hơn nhiều so với SRAM nội. Project dùng SDRAM từ `0xD0000000` và đặt buffer thứ hai tại offset `0x50000`. Đây là quyết định kiến trúc bắt buộc, không chỉ là tối ưu.

5.4.3 Xé hình và đồng bộ VBLANK

Nếu đổi layer ngay khi LTDC đang quét, một phần màn hình thuộc frame cũ và phần còn lại thuộc frame mới. Reload ở vertical blanking giải quyết vấn đề; timeout 25 ms bảo vệ hệ thống nếu cờ VBR không tự xóa như dự kiến.

5.4.4 Tọa độ cảm ứng phụ thuộc orientation

Touch panel điện trở có thể đảo hoặc hoán đổi trục theo revision. Firmware hiện kiểm tra nhiều phép biến đổi khi hit-test. Giải pháp hoạt động thực dụng nhưng làm tăng số phép so sánh. Phương án tốt hơn là hiệu chỉnh ba điểm và chuẩn hóa tọa độ ngay tại driver.

5.4.5 Cân bằng ISR và superloop

DMA audio cần phản hồi đều nhưng sinh hàng nghìn mẫu trong ISR sẽ chặn các ngắt khác. Firmware dùng ISR để đặt cờ và sinh mẫu trong main. Đổi lại, superloop phải không được block quá lâu; mọi chức năng mới cần tuân thủ nguyên tắc này.

5.5 Hạn chế của phiên bản hiện tại

- High score chỉ lưu trong RAM, mất sau reset.
- Chưa có đo tải CPU, jitter TIM6, thời gian render và audio underrun bằng DWT/logic analyzer.
- Âm thanh ở 8 kHz và dạng sóng đơn giản, chất lượng phù hợp game retro nhưng không cao.
- Touch hit-test dùng nhiều biến đổi orientation thay vì một phép hiệu chỉnh thống nhất.
- Mã nguồn chạy superloop; khi mở rộng nhiều tác vụ, việc bảo đảm deadline sẽ khó hơn.
- Báo cáo chưa có ảnh chụp oscilloscope hoặc ảnh board thực tế, do đó các kết luận về tín hiệu vật lý cần được kiểm chứng khi nghiệm thu.

6. KẾT LUẬN

6.1 Kết quả đạt được

Hệ thống đã tích hợp nhiều khối ngoại vi của STM32F429 trong một ứng dụng tương tác có yêu cầu thời gian thực. FMC và SDRAM giải quyết giới hạn bộ nhớ hiển thị; LTDC và DMA2D hỗ trợ xử lý đồ họa; TIM6 tạo cơ sở thời gian; EXTI và I2C3 tiếp nhận dữ liệu đầu vào; I2S3 và DMA1 duy trì luồng âm thanh liên tục. Kiến trúc mô-đun bảo đảm mức độ độc lập tương đối giữa thuật toán trò chơi và trình điều khiển phần cứng.

Về phần mềm, game có máy trạng thái đầy đủ, ba mức độ khó, bốn skin, bốn theme, pause/retry/menu, nhạc nền và hiệu ứng. Vật lý và va chạm đủ nhẹ cho Cortex-M4; renderer vector tránh lưu bitmap lớn; double buffering đồng bộ VBLANK hướng tới chuyển động mượt và không xé hình.

Kết quả ELF cho thấy dung lượng Flash và SRAM tĩnh còn nhiều dư địa. Phần tài nguyên lớn nhất là framebuffer, đã được đặt đúng trong SDRAM ngoài. Thiết kế ISR đặt cờ và xử lý nặng ở superloop cũng tạo nền tảng tốt để kiểm soát thời gian đáp ứng.

6.2 Hướng phát triển

Các hướng cải tiến ưu tiên gồm:

1. Dùng DWT cycle counter và GPIO marker để đo thời gian render, tải CPU, jitter và deadline audio.
2. Lưu high score và thiết lập âm thanh vào Flash với wear leveling đơn giản.
3. Chuẩn hóa touch bằng ma trận affine hiệu chỉnh, thay cho nhiều phép đảo/hoán đổi khi hit-test.
4. Bổ sung nhiều hiệu ứng âm thanh, envelope ADSR hoặc dữ liệu PCM/ADPCM ngắn nếu Flash cho phép.
5. Tách renderer thành dirty-region hoặc sprite cache để giảm số pixel phải vẽ mỗi frame.
6. Thêm watchdog và cơ chế phục hồi nếu I2C, LTDC hoặc DMA gặp lỗi kéo dài.
7. Xây dựng bộ test logic game trên host cho LCG, tính điểm, va chạm và chuyển trạng thái.

6.3 Lời cảm ơn

Nhóm xin cảm ơn giảng viên hướng dẫn và bộ môn đã cung cấp kiến thức, thiết bị và môi trường thực hành. Nhóm mong nhận được góp ý để hoàn thiện việc đo kiểm trên phần cứng, nâng chất lượng âm thanh và cải thiện khả năng bảo trì của firmware.

A. KỊCH BẢN CHỤP ẢNH VÀ NGHIỆM THU

Phụ lục này quy định bộ ảnh tối thiểu để chứng minh hệ thống hoạt động trên phần cứng thật. Nhóm đặt ảnh vào thư mục `figures/screenshots` với đúng tên trong Bảng A.1; khi biên dịch lại, các khung giữ chỗ sẽ tự động được thay bằng ảnh thật.

A.1 Danh sách cảnh cần ghi nhận

Bảng A.1: Kịch bản chụp ảnh và ảnh màn hình nghiệm thu

STT	Tên tệp	Chuẩn bị/thao tác	Nội dung cần nhìn rõ
1	01-board-overview.jpg	Cấp nguồn, nối loa và mở menu chính.	Board STM32F429I-DISC1, LCD, module MAX98357, loa và dây nối.
2	02-cubemx-clock-tree.png	Mở tab Clock Configuration.	HSI, PLL và SYSCLK 180 MHz; clock AHB/APB.
3	03-cubemx-pinout.png	Mở Pinout & Configuration.	PA0, PG2, I2S3, I2C3, LTDC và FMC đã cấu hình.
4	04-build-success.png	Clean Project rồi Build.	Tên project, console kết thúc và số lỗi bằng 0.
5	05-main-menu.jpg	Reset board, chờ menu xuất hiện.	Menu chính rõ nét, màu sắc đúng, không xé hình.
6	06-gameplay.jpg	Bắt đầu game và chơi qua ít nhất một cột.	Chim, hai cột, nền, mặt đất, HUD và điểm số.

Bảng A.1: Kịch bản chụp ảnh và ảnh màn hình nghiệm thu (tiếp theo)

STT	Tên tệp	Chuẩn bị/thao tác	Nội dung cần nhìn rõ
7	07-settings-collage.png	Chụp ba màn Bird, Difficulty, Sound rồi ghép.	Các lựa chọn skin, độ khó, nhạc/SFX và volume.
8	08-pause-gameover.png	Chụp trạng thái Pause và Game Over rồi ghép.	Nút Continue/Menu, điểm hiện tại và high score.
9	09-input-demo.jpg	Chạm LCD hoặc nhấn USER trong lúc chơi.	Tay người dùng và phản hồi trên LCD trong cùng khung hình.

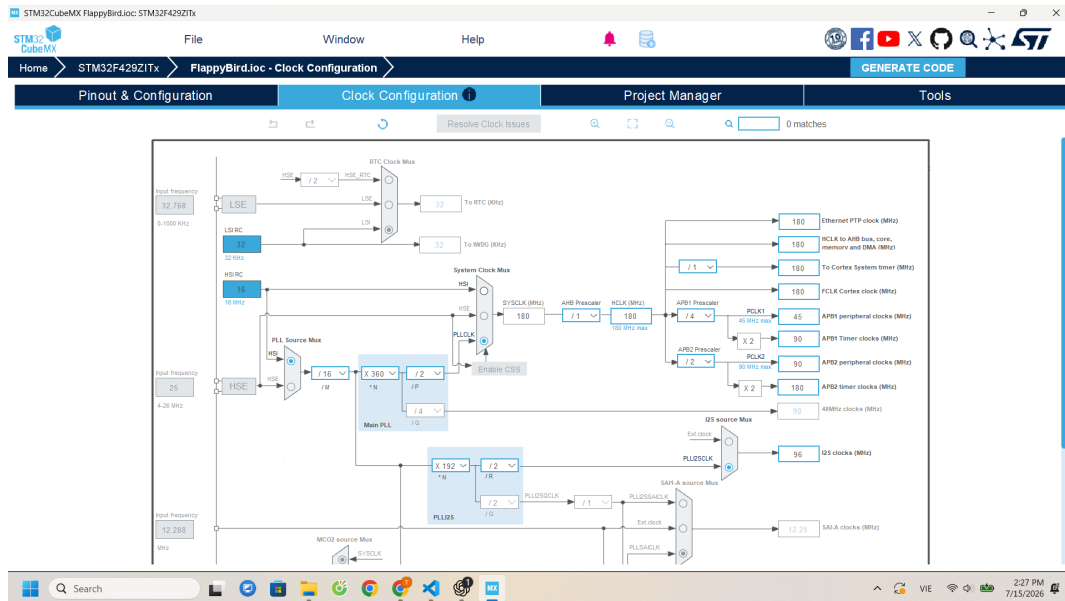
A.2 Khung ảnh sẽ dùng trong báo cáo

ẢNH SẼ ĐƯỢC BỔ SUNG KHI NGHIỆM THU

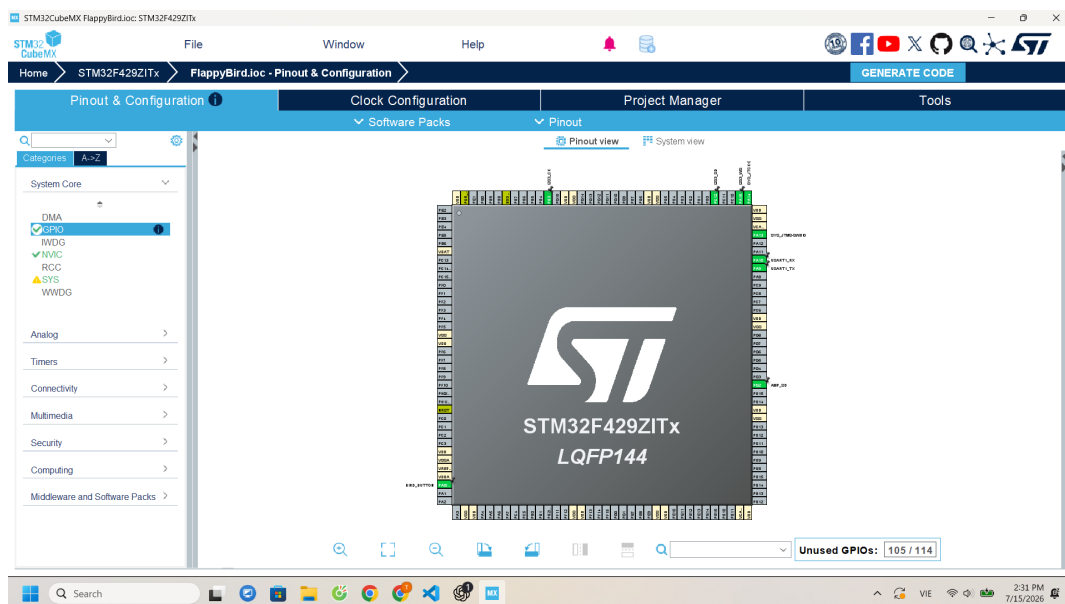
Đặt ảnh đúng tên: `figures/screenshots/01-board-overview.jpg`

Chụp góc nghiêng nhẹ, đủ sáng, thấy rõ board, module khuếch đại và loa.

Hình A.1: Mô hình phần cứng hoàn chỉnh của hệ thống



Hình A.2: Cây clock cấu hình trên STM32CubeMX



Hình A.3: Cấu hình chân vi điều khiển trên STM32CubeMX

ẢNH SẼ ĐƯỢC BỔ SUNG KHI NGHIỆM THU

Đặt ảnh đúng tên: figures/screenshots/04-build-success.png

Giữ phần cuối Console và Problems để chứng minh project không có lỗi biên dịch.

Hình A.4: Kết quả biên dịch thành công trên STM32CubeIDE

ẢNH SẼ ĐƯỢC BỔ SUNG KHI NGHIỆM THU

Đặt ảnh đúng tên: `figures/screenshots/05-main-menu.jpg`

Chụp vuông góc với LCD, giảm phản chiếu và lấy nét vào chữ/nút.

Hình A.5: Màn hình menu chính trên LCD

ẢNH SẼ ĐƯỢC BỔ SUNG KHI NGHIỆM THU

Đặt ảnh đúng tên: `figures/screenshots/06-gameplay.jpg`

Chụp khi đang có đủ chim, cột, HUD và điểm số khác 0.

Hình A.6: Giao diện trong khi chơi FlappyBird

A.3 Quy tắc trình bày ảnh

Ảnh phải là kết quả của chính mô hình nhóm, không dùng ảnh minh họa trên Internet. Mỗi ảnh cần có một đối tượng kiểm chứng rõ ràng; tránh chụp quá rộng khiến chữ trên LCD hoặc công cụ không đọc được. Với ảnh đo tín hiệu, phải giữ tên kênh, thang đo và giá trị tần số. Trước khi nộp, nhóm thay toàn bộ khung giữ chỗ, kiểm tra lại Danh mục hình vẽ và bảo đảm chú thích ảnh khớp với nội dung thực tế.

Tài liệu tham khảo

- [1] STMicroelectronics, *STM32F427xx and STM32F429xx Datasheet*, STMicroelectronics.
- [2] STMicroelectronics, *RM0090 Reference Manual: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs*.
- [3] STMicroelectronics, *UM1670: Discovery kit with STM32F429ZI MCU*.
- [4] STMicroelectronics, *STM32CubeF4 HAL and Low-Layer Drivers*, phiên bản được tích hợp trong project.
- [5] Ilitek, *ILI9341 a-Si TFT LCD Single Chip Driver Datasheet*.
- [6] STMicroelectronics, *STMPE811 8-bit port expander with advanced touch-screen controller Datasheet*.
- [7] Analog Devices / Maxim Integrated, *MAX98357 PCM Input Class D Audio Power Amplifier Datasheet*.
- [8] Nhóm thực hiện, *Mã nguồn project FlappyBird STM32F429ZI*, <https://github.com/TamKudo/embedded-project-2025.2>.